1. Project Goal
Build a standalone handheld FPGA computer using a custom 8-bit CPU, custom instruction set, display, buttons, PCB and enclosure.

2. CPU Architecture
Data width: 8-bit
Address width: 16-bit
Registers: 4 general-purpose registers
Instruction width: 16-bit
ALU: 8-bit
Architecture: custom CPU implemented in FPGA

3. Registers
R0 - General-purpose 8-bit register
R1 - General-purpose 8-bit register
R2 - General-purpose 8-bit register
R3 - General-purpose 8-bit register

PC - 16-bit Program Counter
Stores the address of the next instruction.

SP - 16-bit Stack Pointer
Points to the current location of the stack in memory.

FLAGS - Status register
Stores:
Z - Zero flag
C - Carry flag
N - Negative flag

4. Instruction Set
LOAD Load a value from memory into a register
STORE Store a register value into memory
MOV Copy a value between registers
ADD Add two values
SUB Subtract two values
AND Bitwise AND
OR Bitwise OR
XOR Bitwise XOR
CMP Compare two values and update flags
JMP Jump to another instruction
JZ Jump if Zero flag is set
JNZ Jump if Zero flag is not set
PUSH Push a value onto the stack

POP Remove a value from the stack
IN Read input from an input device
OUT Send data to an output device
HALT Stop CPU execution
LOADI Load an immediate value into a register

5. Memory Model
ByteForge uses separate program memory and data memory.

Program memory stores CPU instructions and is addressed by the 16-bit Program Counter.

The data-memory address space is 16-bit, allowing up to 65,536 addressable locations. This space contains RAM and memory-mapped input/output devices.

The stack is stored inside RAM. Certain addresses in the data-memory space will be reserved for hardware such as the buttons and display.

6. Input/Output
Inputs:
- Directional buttons
- Action buttons
- Possibly additional system buttons

Outputs:
- Small colour display
The CPU will communicate with the input and output hardware using memory-mapped I/O.
Reading from specific addresses will allow the CPU to determine which buttons are pressed.
Writing to specific addresses will allow the CPU to control the display.

7. Final Success Criteria
ByteForge V1 will be considered successful if:
1. The custom 8-bit CPU runs correctly on an FPGA.
2. The CPU can execute programs using the custom instruction set.
3. Programs can read physical button inputs.
4. Programs can output graphics or text to the display.
5. The CPU and supporting hardware are integrated onto a custom PCB.
6. The device is powered and programmable through an appropriate external connection such as USB-C.
7. The electronics fit inside a custom enclosure.
8. A simple assembler can convert ByteForge assembly code into machine code.
9. The finished computer can run multiple demonstration programs, including:
   - an arithmetic/calculator demonstration
   - an interactive input/output demonstration
   - a simple game such as Pong or Snake

8. Instruction Encoding

ByteForge uses 16-bit instruction words.

The first 5 bits are used for the opcode. This gives up to 32 possible instruction types, which is enough for the current instruction set while leaving room for future expansion.

Because ByteForge has four general-purpose registers, each register can be identified using 2 bits:

00 = R0

01 = R1

10 = R2

11 = R3

Different instructions use the remaining bits differently depending on whether they operate on registers, immediate values, or memory addresses.

For register-based instructions such as ADD R1, R2, the instruction contains the opcode and the two register selections in the same 16-bit word.

For immediate instructions such as LOADI R0, 25, the instruction contains the opcode, destination register and an 8-bit constant value.

Instructions that need a full 16-bit memory address use a second 16-bit word immediately after the main instruction. For example:

LOAD R1, [0x2000]

would use one word for the LOAD instruction and register selection, followed by a second word containing the address 0x2000.

OPCODE TABLE

BINARY	INSTRUCTION	PURPOSE
00000	LOAD	Load a value from memory into a register
00001	STORE	Store a register value into memory
00010	MOV	Copy a value between registers

00011	LOADI	Load an immediate value into a register
00100	ADD	Add two register values
00101	SUB	Subtract one register value from another
00110	AND	Perform bitwise AND
00111	OR	Perform bitwise OR
01000	XOR	Perform bitwise XOR
01001	CMP	Compare two register values and update flags
01010	JMP	Jump to another instruction
01011	JZ	Jump if the Zero flag is set
01100	JNZ	Jump if the Zero flag is not set
01101	PUSH	Push a register value onto the stack
01110	POP	Pop a value from the stack into a register
01111	IN	Read data from an input device
10000	OUT	Send data to an output device
10001	HALT	Stop CPU execution

The remaining opcode values are left unused so more instructions can be added later if needed.

9. CPU Datapath

The CPU datapath describes how data moves between the main parts of the ByteForge processor.

The Program Counter stores the address of the next instruction to be executed.

That address is sent to Program Memory, which returns the instruction stored at that location.

The instruction is placed into the Instruction Register, then passed to the Instruction Decoder.

The Instruction Decoder reads the opcode and register fields. The Control Unit then determines which parts of the CPU need to be activated.

The four general-purpose registers, R0 to R3, can provide values to the 8-bit ALU.

The ALU performs arithmetic and logic operations such as addition, subtraction, AND, OR and XOR. The result can then be written back into one of the registers.

The ALU also updates the FLAGS register, including the Zero, Carry and Negative flags.

The CPU communicates with RAM and memory-mapped input/output devices through the Memory Interface.

This allows programs to read physical button inputs and send data to the display.

10. Fetch-Decode-Execute Cycle

ByteForge will execute instructions using a fetch-decode-execute cycle.

Fetch:
The Program Counter contains the address of the next instruction. This address is sent to Program Memory, and the instruction is loaded into the Instruction Register.

Decode:
The Instruction Decoder reads the opcode and register fields. The Control Unit determines what operation is required and which parts of the CPU need to be activated.

Execute:
The CPU carries out the instruction. This could involve performing an ALU operation, reading or writing memory, moving data between registers, accessing an input/output device, or changing the Program Counter for a jump instruction.

After the instruction is completed, the Program Counter moves to the next instruction unless the current instruction changes it.

11. Memory Architecture Refinement

ByteForge will use separate program memory and data memory.

Program memory stores the instructions executed by the CPU and is addressed using the Program Counter.

Data memory contains RAM and memory-mapped input/output devices.

The stack is stored inside RAM and is controlled by the 16-bit Stack Pointer.

Separating program memory from data memory simplifies the CPU design because instruction fetching and data access can be handled independently.

12. Stack Operation

The Stack Pointer stores the address of the current top of the stack.

The stack is stored inside RAM.

PUSH:

1. The Stack Pointer is decremented.

2. The selected register value is written to the RAM address pointed to by the Stack Pointer.

POP:

1. The value at the RAM address pointed to by the Stack Pointer is loaded into the selected register.

2. The Stack Pointer is incremented.

The stack provides temporary storage for programs and can later support subroutines if CALL and RET instructions are added in a future version.

13. ALU Implementation and Testing

ByteForge's 8-bit ALU has been implemented in Verilog. It takes two 8-bit inputs and a 5-bit opcode, and produces an 8-bit result with Zero, Carry and Negative flag outputs. The ALU is combinational and does not require a clock.

It supports ADD, SUB, AND, OR, XOR and CMP. CMP produces the same subtraction result and flags as SUB; the CPU control unit will later prevent that result from being written into a register.

The Zero flag is set when the result is zero. The Negative flag follows bit 7 of the result. For ADD, Carry indicates that the unsigned sum exceeds 255. For SUB and CMP, Carry indicates an unsigned borrow, when A is less than B. Logic operations clear Carry. Unsupported opcodes produce a zero result with all flags cleared.

The ALU was simulated using Icarus Verilog on EDA Playground. All 38 test cases passed with zero failures, covering arithmetic, bitwise operations, flag behaviour and unsupported opcodes.

Simulation: https://www.edaplayground.com/x/R8vK

14. Register File, FLAGS and Reset

ByteForge's register file and FLAGS storage have been implemented in Verilog.

The register file contains four writable 8-bit registers, R0–R3. Two combinational read ports allow two register values to be read simultaneously without waiting for a clock edge. A single write port updates the selected register on the rising clock edge when write_enable is set. Otherwise, the registers retain their stored values.

The FLAGS module stores the Zero, Carry and Negative flags. These update on the rising clock edge when flags_write_enable is set; otherwise, their previous values are preserved. Register writes and flag updates have independent enables, allowing them to occur separately or together.

Both modules use an active-high synchronous reset. When reset is asserted, the next rising clock edge clears all four registers and the three stored flags to zero. Reset takes priority over writing.

The modules were tested using a self-checking testbench in Icarus Verilog on EDA Playground. All 131 checks passed with zero failures. The checks covered register reads and writes, value retention, independent flag updates, simultaneous updates, and synchronous reset behaviour.

Simulation: https://www.edaplayground.com/x/ADVd

15. Instruction Encoding and Decoder Rules

ByteForge uses 16-bit instruction words. The opcode occupies bits 15–11. Register selections use two bits: 00 = R0, 01 = R1, 10 = R2, and 11 = R3.

Instruction formats

Format	Bits 15-11	Bits 10-9	Bits 8-7	Remaining bits
Two Register	Opcode	Register A	Register B	Bits 6-0 reserved
Immediate	Opcode	Destination register	Part of immediate	Immediate occupies bits 8-1; bit 0 reserved
Single register	Opcode	Selected Register	Reserved	Bits 8-0 reserved
Address only	Opcode	Reserved	Reserved	Bits 10-0 reserved
HALT	Opcode	Reserved	Reserved	Bits 10-0 reserved

All reserved bits must be zero in valid ByteForge V1 instructions.

Instruction groups and lengths

Instructions	Format	Length
MOV, ADD, SUB, AND, OR, XOR, CMP	Two register	1 word
LOADI	Immediate	1 word
PUSH, POP	Single register	1 word
LOAD, STORE, IN, OUT	Single register followed by a 16 bit address	2 words
JMP, JZ, JNZ	Address only followed by a 16 bit target	2 words
HALT	HALT	1 word

The opcode values remain those defined in Section 8.

Register and operand meaning

For MOV, Register A is the destination and Register B is the source. For arithmetic and logic operations, Register A supplies the first operand and receives the result; Register B supplies the second operand. SUB calculates A − B.

CMP calculates A − B and updates flags without changing either register.

LOAD, LOADI, IN and POP write to the selected register. STORE, OUT and PUSH read from the selected register.

IN and OUT use the same data-memory address space as LOAD and STORE, allowing access to memory-mapped devices.

Program addressing

Program memory is addressed in 16-bit words. A one-word instruction advances the Program Counter by one; a two-word instruction advances it by two, unless a taken jump replaces the Program Counter.

The second word contains the complete address or jump target. Jump targets are program-word addresses; LOAD, STORE, IN and OUT addresses refer to 8-bit locations in data memory.

Decoder behaviour

The decoder is combinational and requires no clock or reset. It extracts the opcode, relevant register selections and immediate value, and indicates whether a second address word is required.

It marks unsupported opcodes or nonzero reserved bits as invalid. Invalid instructions must not request register writes, flag updates, memory operations, jumps or HALT.

The decoder identifies the instruction; the later control unit handles fetching the second word and executing the operation.

16. Instruction Decoder Implementation and Testing

ByteForge's instruction decoder was implemented in Verilog according to the encoding rules in Section 15. It reads the first 16-bit instruction word, identifies the opcode, extracts register selections and immediate values, and indicates whether a second word is required.

The decoder produces intent signals for register writes, flag updates, memory reads and writes, jumps, stack operations and HALT. These signals describe the actions required by an instruction. A future clocked control unit will determine when those actions occur.

Unsupported opcodes and instructions with nonzero reserved bits are marked invalid. For invalid instructions, all decoded fields and intent signals are cleared, while the raw opcode remains available. The decoder is combinational, with default assignments on every path to prevent inferred latches.

The self-checking testbench was run using Icarus Verilog in EDA Playground. It completed 1,781 checks with zero failures. Testing covered all 18 supported opcodes, every two-register combination, all 256 immediate values for each destination register, single-register selections, jump conditions, unsupported opcodes, reserved-bit violations and transitions between valid and invalid instructions.

The simulation generated instruction_decoder_waveform.vcd. The source files are instruction_decoder.v and instruction_decoder_tb.v, and the results screenshot is saved as ByteForge_Decoder_Test_Results.png.

Saved simulation: ByteForge Instruction Decoder Tests.

17. Control Unit and Instruction Execution Rules

ByteForge will use a clocked control unit to coordinate instruction fetching, decoding and execution. The control unit will convert the decoder's intent signals into timed actions for the ALU, registers, FLAGS, program memory, data memory, Program Counter and Stack Pointer. Only one instruction will execute at a time.

Clock and reset

State changes and stored-value updates occur on the rising clock edge. Reset is active-high and synchronous, taking priority over execution. Reset clears the Program Counter to 0x0000, clears the registers and FLAGS, cancels pending CPU requests, and returns the control unit to instruction fetching.

The Stack Pointer resets to a configurable STACK_TOP value. The exact value and permitted stack address range will be fixed when the physical RAM capacity and final memory map are selected, before CPU integration.

Memory interface and timing

Program and data memory use a request/ready interface. The CPU asserts a request and holds the address, operation and write data stable until memory asserts ready. A transaction completes on a rising clock edge when both request and ready are high.

For reads, memory must provide valid read data at that completion edge. For writes, memory must perform the write exactly once at that edge. The CPU then leaves the waiting state and removes the request. This allows memory operations to take one or more clock cycles.

While waiting for memory, the CPU does not repeat register writes, flag updates or Stack Pointer changes.

Instruction fetching

The control unit requests the 16-bit program word at the Program Counter. When the read completes, it stores the word in the Instruction Register and increments the Program Counter by one.

The instruction is then decoded. If it requires a second word, the CPU requests the word at the updated Program Counter, stores it in an Address Register, and increments the Program Counter again.

Consequently, before execution, the Program Counter points to the next sequential instruction. A taken jump replaces this value with its target address.

Register and ALU instructions

MOV copies Register B into Register A. LOADI writes the decoded 8-bit immediate value into the selected register. Neither instruction changes FLAGS.

ADD, SUB, AND, OR and XOR pass the selected register values to the ALU. The result is written into Register A, and Z, C and N are stored in FLAGS on the same rising edge.

CMP uses the ALU's subtraction operation and updates FLAGS without writing either register. Each instruction performs its register and flag updates once, then returns to instruction fetching.

Data-memory instructions

LOAD and IN read one byte from the address held in the Address Register. When the read completes, the byte is written into the selected register.

STORE and OUT write the selected register's byte to that address. They complete when memory acknowledges the write.

These instructions preserve FLAGS. IN and OUT use the same data-memory interface as LOAD and STORE; peripheral selection is handled by the memory map.

Jump instructions

JMP always loads the Address Register's target into the Program Counter.

JZ loads the target only when the stored Zero flag is one. JNZ loads it only when the stored Zero flag is zero. If the condition is false, the Program Counter retains the next sequential instruction address.

Jump instructions preserve registers and FLAGS.

Stack instructions

The stack grows towards lower RAM addresses.

PUSH calculates SP − 1 and writes the selected register's byte to that address. When the write completes, the Stack Pointer is updated to the calculated address.

POP reads the byte at the current Stack Pointer. When the read completes, the byte is written into the selected register and the Stack Pointer increments by one.

Both instructions preserve FLAGS. Stack addresses must remain within the configured stack region. A PUSH beyond its lower boundary or a POP from an empty stack causes a fault before any memory transaction or stored-value update occurs.

HALT and invalid instructions

HALT enters a stopped state. The CPU issues no further memory requests or register, flag, Program Counter or Stack Pointer updates until reset.

An unsupported opcode or nonzero reserved bit causes the CPU to enter a fault state and assert a fault indicator. The invalid instruction performs no execution actions. Reset clears the fault and restarts instruction fetching.

Control states

The control unit will use states for instruction fetch, decode, second-word fetch, execution, data-memory access, HALT and fault handling. Additional states may separate address preparation from memory transactions. Write enables default to zero and are asserted only for the cycle in which an update is required.

These rules define the intended behaviour. The next step is to implement and test the control unit before connecting it to the complete CPU.

18. Control Unit Implementation and Testing

ByteForge's clocked control unit was implemented in Verilog according to the execution rules in Section 17. It uses a finite-state machine to fetch instructions, decode them, fetch second words where required, and coordinate execution. It stores the Program Counter, Stack Pointer, Instruction Register, Address Register and memory-transaction information.

The control unit instantiates the previously tested instruction decoder. It generates timed register and FLAGS write enables, controls memory requests, and handles conditional jumps, PUSH, POP, HALT and faults. Memory requests and their payloads remain stable while waiting for acknowledgement. Updates occur once at the appropriate completion edge.

Stack boundaries are configurable. PUSH checks for available space before requesting a write, while POP checks that the stack is not empty before requesting a read. Invalid instructions and stack boundary violations enter a fault state. HALT and fault states remain stopped until reset.

A self-checking testbench was run using Icarus Verilog in EDA Playground. It completed 73 scenarios and 10,577 checks with zero failures. Testing covered instruction execution, register and flag updates, program addressing, jumps, memory delays, stack behaviour, invalid instructions, HALT, faults and reset during pending transactions.

This stage used behavioural models of the register file, ALU, FLAGS and memories. Integration with the actual previously tested CPU modules is the next stage.

The source files are control_unit.v and control_unit_tb.v, used alongside instruction_decoder.v. The simulation generated control_unit_waveform.vcd, and the results screenshot is saved as ByteForge_Control_Unit_Test_Results.png.

Saved simulation: ByteForge Control Unit Tests.

19. CPU Integration and Testing

ByteForge's previously tested modules were connected into a CPU core using cpu.v. The wrapper connects the control unit, register file, FLAGS register and ALU. The instruction decoder remains instantiated inside the control unit. The wrapper adds no execution logic or architectural storage.

The register file supplies operands to the ALU and control unit. ALU results and flags return to the control unit, which controls register writes and FLAGS updates. The stored Zero flag feeds conditional jump decisions. Program and data memory remain external to the CPU core.

The integration testbench, cpu_tb.v, uses the actual CPU design modules with behavioural program and data memories. It was run using Icarus Verilog in EDA Playground and completed 75 scenarios and 15,234 checks with zero failures.

Testing covered arithmetic and logic, register transfers, flag behaviour, memory accesses, conditional jumps, a backward-branch loop, stack operations, invalid instructions, HALT, faults and reset. Memory delays and transaction checks verified that requests remain stable and operations complete once.

A mixed program combined arithmetic, comparison, branching, memory and stack operations before HALT. Its final register values, FLAGS, Program Counter, Stack Pointer and selected memory bytes matched the expected results.

This confirms CPU-core operation in simulation. FPGA memory implementation, display and button interfaces, and physical hardware testing remain future stages.

The simulation generated cpu_waveform.vcd. The results screenshot is saved as ByteForge_CPU_Test_Results.png.

Saved simulation: ByteForge CPU Integration Tests.

20. Assembler and Program Testing Rules

ByteForge will use a Python assembler to convert readable assembly instructions into 16-bit machine-code words. Its output must follow the opcode values and instruction formats defined in Sections 8 and 15.

Assembly syntax

Each line may contain an instruction, a label or a comment. Instructions and register names are case-insensitive. Commas separate operands, and a semicolon starts a comment.

Examples:

LOADI R0, 10

MOV R1, R0

ADD R0, R1

STORE R0, [0x0200]

HALT

Only R0, R1, R2 and R3 are valid registers. Numbers may use decimal, hexadecimal with 0x, or binary with 0b. Immediate values must be between 0 and 255. Addresses must be between 0 and 65535. Out-of-range values must produce errors rather than being truncated.

Supported instruction forms

Instructions	Operands
MOV, ADD, SUB, AND, OR, XOR, CMP	Two registers
LOADI	Register and 8-bit immediate
LOAD, STORE, IN, OUT	Register and address in square brackets
PUSH, POP	One register
JMP, JZ, JNZ	Numeric target or label
HALT	No operands

Labels and program addresses

A label ends with a colon and identifies a program-word address. Programs begin at address zero. One-word instructions advance the assembly address by one; two-word instructions advance it by two.

The assembler uses two passes: the first records label addresses, and the second encodes instructions and resolves jump targets. Forward and backward jumps must both work.

Example:

LOADI R0, 3

LOADI R1, 1

loop:

SUB R0, R1

JNZ loop

HALT

Encoding and output

The assembler must set every reserved bit to zero. Instructions requiring a second word must emit the complete 16-bit address or jump target immediately after the first word.

It produces a text .hex file containing one four-digit hexadecimal word per line, suitable for loading into a Verilog program-memory model using $readmemh. It also produces a listing showing each instruction’s program address and encoded words.

Error handling

Unknown instructions, invalid registers, incorrect operand counts, malformed numbers, duplicate labels, undefined labels and out-of-range values must produce clear errors with source-line numbers. Assembly failure must return a nonzero exit status and must not create a partial machine-code file.

Testing

Assembler tests will check all 18 supported instructions, operand boundaries, reserved bits, label resolution and invalid inputs against independently specified expected encodings.

Three demonstration programs will cover arithmetic, a countdown loop, and memory/stack operations. Their assembled .hex files will be loaded into a CPU simulation using the existing CPU modules. The testbench will verify expected register values, FLAGS, relevant memory locations and HALT without fault.

This stage tests the assembler and CPU together in simulation. Physical display, button and FPGA memory interfaces remain later work.

21. Assembler Implementation and Program Testing

ByteForge's assembler was implemented in Python according to Section 20. It converts assembly instructions into 16-bit machine-code words using the existing instruction encoding. Two passes allow it to record label addresses before resolving forward and backward jumps.

The assembler supports all 18 instructions, R0–R3, decimal, hexadecimal and binary numbers, and semicolon comments. It validates operands, reserved bits and value ranges, reporting invalid input with source-line numbers. It generates a .hex machine-code file and a .lst listing showing instruction addresses and encoded words.

The Python test suite completed 78 tests with no failures. Testing covered instruction encodings, register combinations, immediate and address boundaries, labels, output formatting, invalid inputs and command-line behaviour.

Three demonstration programs were assembled: arithmetic.asm, countdown.asm and memory_stack.asm. They contain 13, 10 and 16 program words respectively and exercise arithmetic and logic, a conditional countdown loop, and memory transfers with stack operations.

The end-to-end testbench, assembly_programs_tb.v, loads the generated .hex files into behavioural program memory and executes them using the actual CPU modules. Each program was tested with immediate-ready, fixed-delay and random-delay memory responses.

The EDA Playground simulation completed nine runs and 1,986 checks with zero failures. All programs reached HALT without a fault, and their final registers, FLAGS, Program Counter, Stack Pointer and checked memory values matched expectations.

This verifies the path from readable assembly through generated machine code to CPU execution in simulation. Physical FPGA memory, display and button interfaces remain future stages.

The simulation generated assembly_programs_waveform.vcd. The results screenshot is saved as ByteForge_Assembly_Program_Test_Results.png.

Saved simulation: ByteForge Assembler and Program Tests.

22. Hardware Selection and Budget

ByteForge V1 will use an FPGA module mounted on a custom carrier PCB, a small colour display, physical buttons and a custom enclosure. The previously tested CPU will run on the FPGA alongside program memory, data RAM and peripheral controllers.

Hardware selection

Component	Selected development target
FPGA module	Sipeed Tang Nano 20K
Display	Waveshare 17344, 2-inch SPI LCD, 240 × 320 pixels
Controls	Four directional buttons and two action buttons
Reset	One separate reset button
Carrier PCB	Custom two-layer board
Enclosure	Custom 3D-printed case
Power and programming	USB through the FPGA module’s USB-C connector

The Tang Nano 20K provides 20,736 LUT4 logic cells and 828 Kbits of block RAM, with onboard programming hardware and configuration flash. It is the selected development target; synthesis must still confirm the complete design's resource usage and timing.

The display uses an ST7789-series controller. An FPGA SPI controller will send commands and pixel data to it through memory-mapped registers. V1 will avoid a full-resolution framebuffer, which would require 153,600 bytes at 16 bits per pixel.

Memory and input handling

The initial implementation target is 8,192 program words of 16 bits each and 16,384 bytes of data RAM. Together, these require 32 KiB before implementation overhead. The CPU retains its 16-bit addresses even though only part of the address space contains physical memory.

Buttons will connect FPGA inputs to ground when pressed, with 10 kΩ pull-up resistors to 3.3 V. FPGA logic will synchronise and debounce the inputs. Button states will be accessible through memory-mapped registers.

PCB and power

USB will power the assembly through the FPGA module's USB-C connector. The carrier PCB will use an AP2112K-3.3TRG1 regulator to convert the Tang Nano's 5 V supply into a separate 3.3 V supply for the display. The regulator will use 1 µF X5R or X7R ceramic capacitors at its input and output. Its output will not connect to the Tang Nano's existing 3.3 V rail. Display control signals will use 3.3 V FPGA pins. GPIO pins will not supply display power. An external USB power bank may provide portable operation; an internal battery and charging circuit are outside V1's scope.

Preliminary budget

The following figures are planning allowances in US dollars, not confirmed supplier quotations.

Category	Allowance
FPGA module	$45
Display and matching cable	$25
Carrier components, headers and mounting hardware	$20

PCB fabrication	$25
Enclosure	$15
USB data cable	$10
Shipping	$50
Tax provision	$17.10
Estimated total	$207.10
Remaining within the $250 planning limit	$42.90

Final supplier prices, delivery costs and manufacturing quotations must replace these allowances before purchasing. An optional external power bank is not included.

The carrier-components allowance includes the display regulator and its two capacitors. Final component prices and manufacturing quotations remain outstanding.

Verification status

CPU and assembler simulation are complete. FPGA synthesis, timing analysis, peripheral implementation and physical testing remain outstanding. The proposed pin assignments must be checked against the board revision and transferred consistently into the schematic and FPGA constraints.

The hardware review package will include the final bill of materials, schematic, routed PCB, manufacturing files, enclosure CAD, firmware and build instructions. This section records the intended hardware configuration and does not claim completed hardware testing or funding approval.

Proposed signal connections

Device signal	FPGA pin	Board label
Display DIN / MOSI	73	IOT40A
Display CLK	74	IOT34B
Display CS	41	IOB43A
Display DC	77	IOT30A
Display RST	48	IOR49B
Display BL	42	IOB42B
Up button	27	IOB8A
Down button	28	IOB8B
Left button	25	IOB6A
Right button	26	IOB6B
Action A button	29	IOB14A

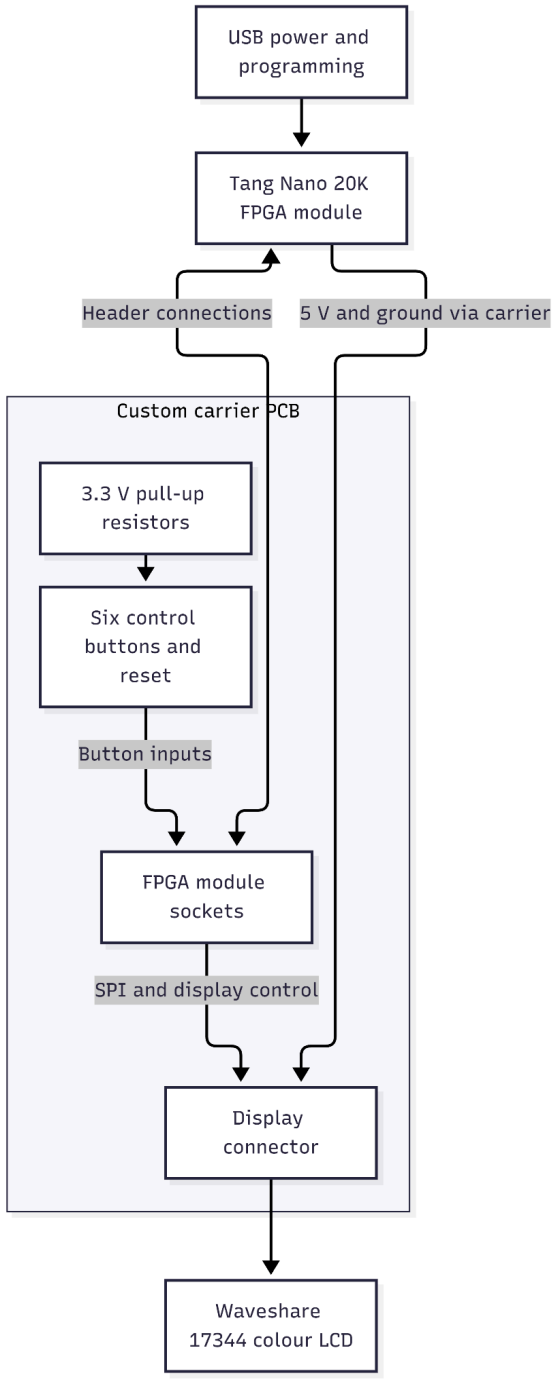
Action B button	30	IOB14B
CPU reset button	31	IOB29A

Power connections

Connection	Destination
USB power	Tang Nano USB-C connector
Tang Nano 5 V header	Carrier display-regulator input
Carrier regulator 3.3 V output	Display VCC
Tang Nano GND	Carrier, regulator and display ground
Tang Nano 3.3 V header	Button pull-up resistors

Each button connects its input to ground when pressed and uses a 10 kΩ pull-up to the Tang Nano's 3.3 V rail. The display's reset signal is separate from the CPU reset button. The board's RGB LCD connector and HDMI port must remain disconnected because some assigned pins are shared with those interfaces. Display CS and RST use FPGA package pins 41 and 48 to avoid the BL616 connections on pins 75 and 86.

These assignments are a planning reference. The schematic and FPGA constraints must use the same mapping, checked against the purchased board revision.



Proposed ByteForge hardware connections: The FPGA module runs the custom CPU, memories and peripheral controllers. The carrier PCB connects the module to the buttons and display. USB provides power and programming access.

The carrier includes an AP2112K-3.3TRG1 regulator and its input/output capacitors to provide a separate 3.3 V supply to the display.

23. Complete FPGA System Implementation and Verification Rules

The next stage will combine the ByteForge CPU, program memory, data RAM, button inputs and display interface into a complete FPGA system targeting the Tang Nano 20K. The existing CPU instruction set and execution behaviour will remain unchanged.

The objective is to complete system simulation and FPGA implementation checks before finalising the KiCad schematic and PCB connections.

System structure

A new system wrapper will instantiate the existing CPU modules and connect them to:

-Program ROM.

-Data RAM and address decoding.

-Button synchronisation and debouncing.

-Memory-mapped display registers and an SPI transmitter.

-A programmable millisecond delay peripheral.

-Clock and reset handling.

The CPU's existing request/ready interfaces will be preserved. Peripheral controllers will operate independently of instruction execution.

Program and data memory

Program memory will contain 8,192 words of 16 bits, addressed from 0x0000 to 0x1FFF. It will be implemented as a synchronous ROM with an initialization file generated from the assembler output. Unused words will contain HALT instructions.

The initialization file will contain exactly 8,192 words. A program-memory request outside the implemented range will return HALT without wrapping the address into valid ROM.

Data RAM will contain 16,384 bytes, addressed from 0x0000 to 0x3FFF. It will use synchronous accesses suitable for FPGA block-memory inference.

Reset will cancel pending transactions but will not clear the complete RAM array. Programs must initialize RAM locations before reading them. Tests must not assume that RAM starts at zero or is cleared by reset.

The ROM initialization mechanism and inferred memory resources must be checked in the selected FPGA toolchain.

Memory map

Program and data addresses belong to separate address spaces.

Address space	Address or range	Function
Program	0x0000–0x1FFF	8,192-word program ROM
Data	0x0000–0x3EFF	General-purpose RAM
Data	0x3F00–0x3FFF	Reserved stack region
Data	0xFF00	Debounced button states, read-only
Data	0xFF10	Display command-byte write
Data	0xFF11	Display data-byte write
Data	0xFF12	Display status, read-only
Data	0xFF13	Backlight control
Data	0xFF20	Millisecond delay request
Data	Other addresses	Unmapped

The CPU will use STACK_BOTTOM = 0x3F00 and STACK_TOP = 0x4000. The stack therefore holds up to 256 bytes. Its existing overflow and underflow protection remains active.

The stack is part of ordinary data RAM. Software must reserve it; LOAD and STORE are not prevented from accessing it.

Unmapped data reads will return zero. Unmapped writes and writes to read-only registers will complete without changing state. They must not alias valid RAM or peripherals.

Transaction behaviour

A transaction completes only on a rising clock edge when request and ready are both high and reset is low.

The system must:

-Hold read data valid at the completion edge.

-Perform each write or peripheral action exactly once.

-Avoid accepting a held request as multiple transactions.

-Require request to return low before accepting another transaction on that interface.

-Suppress ready and side effects while reset is asserted.

-Cancel unfinished transactions on reset.

A peripheral may delay ready while it is busy. The CPU must retain its request and payload until completion.

Buttons and reset

The six control inputs will be active-low physical signals. Each will pass through a two-stage synchronizer and a debouncer. The default debounce interval will be 10 milliseconds, calculated from a configurable system-clock frequency.

The button register will report one for a pressed button:

Bit	Meaning
0	Up
1	Down
2	Left
3	Right
4	Action A
5	Action B
7–6	Always zero

The CPU reset button will remain separate from the button register.

The board-level reset circuit will provide startup reset and handle the external reset button. Reset assertion may be asynchronous in the reset-conditioning circuit, but release must be synchronized to the system clock. The existing CPU, register file and FLAGS modules retain their synchronous reset behaviour.

Display interface

The display interface will target the Waveshare 17344 ST7789-series module.

Writing to 0xFF10 transmits one command byte with DC low. Writing to 0xFF11 transmits one data byte with DC high.

The initial design will use a single-byte transmitter without a FIFO. A display write will wait until the transmitter can accept it. At the bus completion edge, the byte and its command/data selection are captured exactly once.

The SPI transmitter will:

-Use Mode 0 and transmit the most significant bit first.

-Keep clock low when idle.

-Hold DC stable throughout a byte.

-Assert chip selected for the transfer and release it after the final bit.

-Produce exactly eight clock pulses per byte.

-Use a configurable clock divider.

-Abort transmission on system reset and return its outputs to defined idle levels.

At 0xFF12, bit 0 indicates transmission busy and bit 1 indicates display startup/reset timing is in progress. Other bits return zero.

At 0xFF13, bit 0 controls the backlight; other bits are ignored and read as zero. Backlight defaults off after reset.

Display RST is separate from CPU reset. The controller will provide a defined startup reset sequence before accepting display bytes. Exact reset timing, initialization commands, pixel format and orientation will follow the selected module's manufacturer documentation and be recorded with the implementation.

The assembly demonstration program will initialize the display before drawing. CPU reset will restart that initialization sequence.

Delay peripheral

Writing a value from 1 to 255 to 0xFF20 requests that many milliseconds of delay. The peripheral captures the value once, holds the transaction pending, and asserts ready when the interval has elapsed.

Writing zero completes without an intentional delay. Reads return zero. Reset cancels a pending delay.

The delay provides explicit waits for display initialization without relying on software instruction-loop timing.

Clock and FPGA integration

The board wrapper will use one system-clock domain. CPU logic, memories, debouncers and peripheral controllers will use that clock.

SPI and millisecond timing will be generated using counters and clock enables. The SPI output clock will not become an internal fabric clock.

The system-clock frequency must match the actual board clock and timing constraints. The implementation will include device selection, clock constraints and pin assignments consistent with Section 22.

Unused board peripherals will remain disabled or unconnected. Existing onboard connections must be checked for conflicts before assigning pins.

Demonstration program

An assembled program will:

1. Wait for display startup.
2. Send the documented display initialization sequence.
3. Draw a known initial pattern.
4. Read the button register.
5. Change a visible pattern or colour in response to button inputs.

Drawing will stream commands and pixel bytes to the display rather than require a full-screen framebuffer.

The program, generated ROM image and expected display transactions will be included in the repository.

Simulation requirements

Self-checking tests will cover:

-ROM contents, boundaries and out-of-range fetches.

-RAM reads, writes, boundaries and retention across reset.

-Address decoding without unintended aliasing.

-Stack operation with the configured boundaries.

-Button polarity, synchronization, bouncing and stable presses.

-Display status, backpressure and command/data selection.

-SPI bit order, clock pulses, chip select and reset behaviour.

-Delay duration and cancellation.

-Reset during pending memory and peripheral transactions.

-The assembled demonstration program running on the real CPU modules.

A display-monitor model will check transmitted commands and data against independently specified expectations. It must not simply reuse the transmitter's implementation logic.

Tests may shorten configurable timing intervals for simulation, with separate checks confirming the production counter calculations. Every testbench must include a timeout and return a nonzero exit status on failure.

FPGA implementation checks

The complete system will undergo synthesis, placement and routing, and static timing analysis for the selected device.

Results must confirm:

-Successful compilation and implementation.

-Resource usage within device capacity.

-ROM and RAM inferred into appropriate memory resources.

-No unintended latches or unexplained critical warnings.

-Correct clock and I/O constraints.

-Timing closure at the selected system-clock frequency.

Any asynchronous-input timing exceptions must be narrowly applied and documented. Internal timing paths must not be hidden by broad exceptions.

Completion criteria

This stage is complete when system simulations pass, the demonstration program produces the expected display transactions, and the FPGA implementation fits the selected device with timing requirements met.

Simulation does not establish physical display operation, signal integrity, power stability or mechanical fit. Those require later hardware testing.

The resulting source files, assembly program, ROM image, constraints, build instructions and verification reports will support the subsequent KiCad schematic and PCB work.

24. FPGA System Implementation and Verification

ByteForge's CPU was integrated with program ROM, data RAM, memory-mapped peripherals and a Tang Nano 20K board wrapper according to Section 23. The six existing CPU design files remained unchanged.

The system includes an 8,192-word program ROM, 16 KiB of data RAM, six synchronised and debounced button inputs, reset conditioning, a millisecond delay peripheral and an SPI display controller. The memory interface handles synchronous reads, peripheral backpressure and exactly-once writes. RAM contents are preserved across reset.

The display controller transmits command and data bytes using SPI Mode 0. It provides display reset sequencing, status reporting and backlight control. At the 27 MHz system clock, the configured SPI clock is 6.75 MHz.

An assembly demonstration program initializes the display, clears the screen and draws a coloured patch. Button presses trigger changes to the patch. The program streams pixel data without allocating a full-screen framebuffer.

Simulation results

The supplied Icarus Verilog verification logs record a full regression run with zero failures. Coverage included memory boundaries, address decoding, held requests, stack limits, reset cancellation, button bouncing, SPI transfers and peripheral delays.

Test Checks

6. Program ROM: 16,435
7. Data bus and peripherals: 1,413
8. Delay peripheral: 29
9. Button inputs: 58
10. Reset conditioner: 11
11. SPI controller, three divider settings: 172 per setting
12. Stack normal operation, overflow and underflow: 12, 264 and 6
13. Board wrapper: 14
14. Complete demonstration program: 179,499
15. Production timing: 451

The demonstration test used the real CPU modules and an independent SPI monitor to check initialization bytes, pixel transfers, button responses and recovery from two resets. Accelerated timing parameters kept this simulation practical.

A separate production-timing test exercised the real clock-count settings for 10 ms debounce, millisecond delays, display reset phases and SPI timing. Existing regressions also passed: 78 assembler tests with zero skips, 15,234 CPU integration checks and 1,986 assembly-program checks.

Five deliberate mutations were detected by the tests. The regression runner was also corrected so compilation failures produce a nonzero exit status.

FPGA implementation results

The complete design was synthesized, placed and routed using Yosys, nextpnr-himbaechel and Apycula for the Tang Nano 20K's GW2AR-LV18QN88C8/I7 FPGA.

After the display-pin correction, the implementation reports showed:

Item	Result
LUT4 usage reported in the placement log	1,300 of 20,736
Registers	396 of 15,552
Block RAMs	16 of 46
Inferred latches	None reported
Target system clock	27 MHz
Reported maximum internal clock frequency after routing	85.90 MHz
Checked port assignments	16 matched, zero mismatches

A bitstream was generated. Following the pin remap, the shorter regression passed and the generated program files remained identical. The longer demonstration and production-timing tests were not repeated because the RTL and program had not changed.

Hardware corrections

Display CS was moved from FPGA pin 75 to pin 41, and display RST from pin 86 to pin 48, avoiding connections to the onboard BL616. The updated constraints and placement results confirm these assignments.

The carrier PCB will provide a separate 3.3 V display supply using an AP2112K-3.3TRG1 regulator fed from the Tang Nano's 5 V header. Its output will remain separate from the board's existing 3.3 V rail. The RGB LCD connector and HDMI port will remain disconnected.

Verification limits and next stage

The implementation results establish simulation behaviour, FPGA resource fit and internal timing in the open-source flow. External display-interface timing was not analysed, and the Gowin IDE flow remains unverified.

The bitstream has not been loaded onto physical hardware. Display operation, orientation, power stability, startup behaviour and button wiring still require physical testing.

The current implementation is archived as ByteForge_FPGA_2.zip, containing source files, testbenches, assembly software, constraints, build scripts and verification reports. The next stage is the KiCad carrier-board schematic, followed by PCB placement, routing and enclosure design.